PARTE 2 - FOG / CLOUD COMPUTING

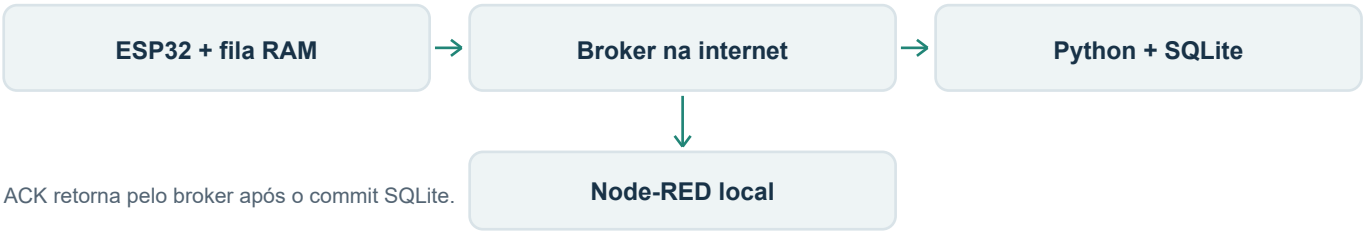
Comunicação MQTT e dashboard Node-RED

Relatório do transporte, persistência, configuração do painel e validação dos alertas.

Grupo Aura | Murilo Salla - RM568041
Elias da Silva de Souza - RM568500 | Julia Duarte de Carvalho - RM567816
FIAP - Inteligência Artificial - Ano 2 | Tutor: Leonardo Ruiz Orabona
Revisão técnica: 28/10/2026 | Evidências: 24/10/2026 | Código de referência: 6fd6aba

1. Arquitetura e distribuição das responsabilidades

Evoluímos o mesmo circuito para transmissão real. O ESP32 coleta e mantém a fila no Edge; o broker público **broker.hivemq.com:1883** intermedeia as mensagens pela internet. Executamos consumidor Python, SQLite e Node-RED no computador local, como camada de processamento e visualização. O painel acompanha o fluxo MQTT; a confirmação de gravação permanece sob responsabilidade do consumidor.



2. Protocolo, identidade e confirmação

Identificamos cada leitura por **(device_id, session_id, sequence)**. O dispositivo é cardioia-esp32-01; uma nova inicialização cria outra sessão. Preservamos tempos relativos do ESP32 e registramos o primeiro recebimento em UTC no consumidor. Incluímos temperatura, umidade, validade do DHT22, pulsos, BPM, perdas, fila, idade e tentativa. Temperatura e umidade inválidas são null, sem substituir ausência por zero.

Usamos a raiz de tópicos **cardioia/aura/fase3/47805457398242816 1/v1**, seguida de **devices/cardioia-esp32-01/**. O número na raiz é um identificador de namespace herdado; a Parte 2 utiliza o projeto Wokwi MQTT indicado nas referências.

Sufixo do tópico	Finalidade e política
telemetry	Leituras publicadas pelo ESP32 com QoS 0, sem retained. Mantemos a amostra na fila até receber seu ACK.
ack	Consumidor publica confirmação com QoS 1, sem retained, somente após validar e concluir a transação SQLite.
status	Estado com retained e Last Will. Distinguimos conexão disponível de leitura recente.

Sem ACK, reenviamos a primeira pendência após três segundos simulados. A chave única no SQLite impede duas linhas para a mesma identidade, inclusive após reiniciar o consumidor. Uma duplicata válida recebe nova confirmação; uma identidade repetida com medição conflitante é rejeitada. Retiramos do ESP32 apenas a cabeça correspondente ao ACK. A garantia combina repetição e deduplicação; não alegamos entrega exatamente uma vez pelo protocolo MQTT.

3. Preparar o computador e a pasta

Os passos abaixo são para Windows. Abrir o menu Iniciar, procurar PowerShell e abrir sem modo administrador. Executar uma linha por vez; se houver erro, corrigir antes de avançar.

1. Instalar: obter Git, Python 3.12 e Node.js 24 com npm nos sites oficiais. Selecionar as versões indicadas. Fechar e reabrir o PowerShell após instalar.

```
git --version

python --version

node --version

npm.cmd --version
```

Esperado: Python 3.12.x, Node v24.x e versões de Git/npm. Se python falhar, testar py -3.12 --version; se funcionar, usar py -3.12 apenas para criar o ambiente abaixo.

2. Baixar: no Explorador de Arquivos, criar C:/projetos ou escolher outra pasta curta com permissão de escrita. No PowerShell, entrar nessa pasta e baixar o código:

```
Set-Location -LiteralPath 'C:\projetos'

git clone https://github.com/murilosalla-blip/fiap-ano02-fase03-cardioia.git

Set-Location -LiteralPath '.\fiap-ano02-fase03-cardioia'

Test-Path .\README.md
```

Esperado: True. Essa é a raiz do projeto. Se a cópia já existe, entrar nela pelo caminho real; não repetir o clone. Caminhos longos podem impedir instalações no Windows.

3. Criar o ambiente: executar a primeira linha somente se .venv ainda não existir. A criação pode terminar sem mensagem. Instalar as bibliotecas e o painel, aguardando cada comando concluir:

```
python -m venv .venv

.\.venv\Scripts\python.exe --version

.\.venv\Scripts\python.exe -m pip install -r config/fase3/requirements-mqtt.txt

npm.cmd ci --prefix src/fase3/node-red --ignore-scripts --no-audit --no-fund
```

Avançar quando: o Python do ambiente indicar 3.12.x e as instalações terminarem sem erro. npm.cmd evita bloqueio do arquivo npm.ps1 no PowerShell. Não é necessário ativar .venv ou comprar hardware.

No Linux/macOS, usar cd, python3.12 -m venv .venv, ./.venv/bin/python e npm. A revisão de reprodução foi feita no Windows. O guia completo detalha versões, alternativas e erros.

4. Executar, visualizar e encerrar

4. Iniciar no terminal A: chamar de A o PowerShell já aberto na raiz. Executar as duas linhas abaixo e manter a janela aberta; o servidor permanece ocupando o terminal.

```
$env:CARDIOIA_PYTHON = (Resolve-Path .\.venv\Scripts\python.exe).Path

node scripts/fase3/iniciar_dashboard.js --com-consumidor
```

5. Abrir o painel: esperar Server now running e Started flows no terminal. A linha inicial Painel: apenas informa o endereço. Então acessar 127.0.0.1:1880/dashboard/monitoramento. Antes da simulação, é normal aguardar dados. Manter apenas um consumidor no tópico.

6. Ligar o circuito: abrir o Wokwi MQTT da Parte 2, clicar em Play e mover a chave à direita. Manter Wokwi e painel visíveis. Esperar ack_confirmado no Serial, avanço da sequência e atualização do painel. O projeto Serial da Parte 1 não alimenta o painel.

7. Testar pulsos: pressionar e soltar o botão verde; seis acionamentos na mesma janela de cinco segundos simulados equivalem a 72 BPM. Para alerta, obter mais de dez acionamentos por janela, em duas coletas atuais seguidas. Depois reduzir para até 108 BPM por duas leituras e conferir retorno. Segurar conta um pulso.

8. Testar temperatura: clicar no DHT22 em execução e ajustar para 31 °C. Aguardar duas leituras atuais e conferir medidor e evento de alerta. Ajustar para 28 °C e aguardar outras duas leituras para conferir o retorno. O sensor mede temperatura ambiente; zero pulsos não indica parada cardíaca.

9. Testar interrupção: mover a chave à esquerda e aguardar três novas coletas; esperar fila crescente. Voltar à direita e conferir pendências em ordem, ACKs e fila até zero. Dados recuperados entram no histórico; não confirmam alertas do presente.

10. Encerrar: clicar em Stop no Wokwi. No terminal A, pressionar Ctrl+C e aguardar o prompt retornar. Preservar .tools/runtime/fase3/: o banco permanece salvo. Reiniciar o painel limpa seu gráfico em memória, sem apagar o SQLite.

5. Como interpretar e resolver dúvidas

- **Sem atualização:** após 15 segundos reais sem leitura atual, o último valor fica cinza. O relógio simulado pode desacelerar em segundo plano.
- **Publicação tentada:** não comprova gravação. Conferir ACK e consumidor; sem ACK, a leitura permanece na fila.
- **Erro ou porta ocupada:** guardar a saída e corrigir antes de continuar. Não iniciar outra cópia do servidor nem apagar o banco.

Usamos Node-RED 5.0.9 e FlowFuse Dashboard 1.34.0, fixados no package-lock. O export flows.json depende de painel.vue, monitor.js e settings.js; o inicializador configura essa ligação. Leituras atuais exigem idade no envio até 10 s, fila de uma posição, conexão e recebimento local nos últimos 15 s. Reenvios não criam novos pontos. Os limites e a confirmação em duas janelas são escolhas didáticas.

6. Validação realizada e evidências

Em 24/10/2026, validamos ESP32, broker, consumidor e retorno do ACK. Na sessão 5f1ffeb88efb4c0c, conferimos **75 medições únicas, sequências 1 a 75, sem lacunas e com zero perdas**. Omitimos o primeiro ACK e identificamos reenvios das sequências 1 e 65 como duplicatas, sem duplicar linhas. Ao parar e reiniciar o consumidor com o mesmo banco, recuperamos as pendências e voltamos à fila vazia.

Também testamos a chave offline e a falha controlada de acesso ao endpoint com broker_off, que direciona somente nosso cliente à porta TCP 1. Restauramos a porta 1883 com broker_on e conferimos recuperação ordenada. Esse ensaio não representa uma indisponibilidade global do HiveMQ. Os validadores dos registros preservados aprovaram ACK omitido, reenvio e recuperação.

Na validação do painel, em outra sessão (546cb527f2cc13d5), conferimos **27 medições únicas**, gráfico, gauge e entrada e retorno dos dois alertas. Registramos 44,3 °C na simulação e depois a redução; na sequência 27, observamos 24,8 °C, umidade de 6,5%, zero pulsos e zero perdas. A consulta ao SQLite complementa as capturas visuais.

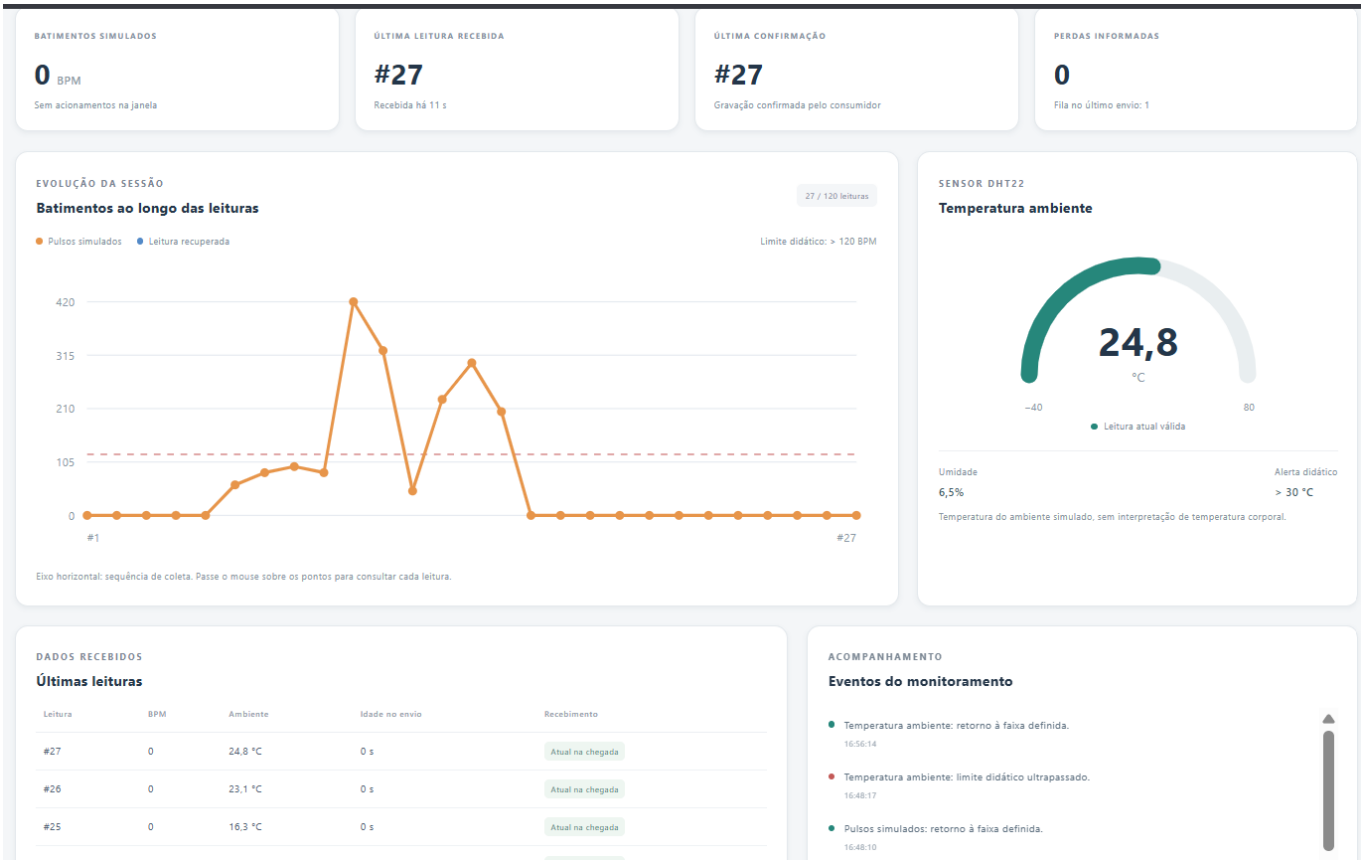


Figura 1. Captura original de 24/10/2026: sequência 27 e retorno da temperatura à faixa definida. O gráfico preserva o histórico de pulsos; zero indica ausência de acionamentos. Fonte: assets/fase3/evidencias/2026-10-24/dashboard/validacao_guiada/03_retorno_temperatura.png.

7. Limites, conclusão e referências

Utilizamos dados fictícios em broker público, sem autenticação ou TLS. A fila do ESP32 é volátil; SQLite preserva somente o que já foi recebido e gravado. O histórico visual em memória reinicia com o painel, sem apagar o banco. Demonstramos transmissão, recuperação e alertas didáticos; não validamos segurança ou uso clínico de um sistema médico.

[1] Wokwi MQTT | Guia de instalação e execução
[2] Firmware, consumidor, painel e export | Contratos e configuração Node-RED
[3] Validação MQTT e conferência das 75 medições
[4] Validação do painel, capturas e 27 medições